

МИНОБРНАУКИ РОССИИ
САНКТ-ПЕТЕРБУРГСКИЙ ГОСУДАРСТВЕННЫЙ
ЭЛЕКТРОТЕХНИЧЕСКИЙ УНИВЕРСИТЕТ
«ЛЭТИ» ИМ. В.И. УЛЬЯНОВА (ЛЕНИНА)
Кафедра САПР

ПРАКТИЧЕСКАЯ РАБОТА №2

по дисциплине

«Алгоритмы и структуры данных»

**Тема: «Реализация JPEG-инспирированного алгоритма сжатия
изображения»**

Студент гр. 4354

Чучалин И.В.

Преподаватель

Пестерев Д.О.

Санкт-Петербург

2026

Задача

Необходимо реализовать JPEG-подобный алгоритм сжатия изображений с использованием дискретного косинусного преобразования, квантования, зигзаг-сканирования, разностного кодирования, RLE и кодирования Хаффмана в соответствии со стандартом ITU-T81. Провести исследование зависимости размера сжатого файла от параметра качества Quality. Обеспечить запись сжатых данных и метаданных в файл, а также корректное декодирование.

Теоретическая часть

1. Цветовые пространства и преобразование YCbCr

Переход из RGB в YCbCr выполняется для разделения яркостной и цветностных компонент. Человеческий глаз менее чувствителен к деталям цветности, что позволяет сильнее сжимать каналы Cb и Cr. Временная сложность преобразования составляет $O(W * H)$, где W и H - ширина и высота изображения, так как каждый пиксель обрабатывается независимо. Пространственная сложность также $O(W * H)$ для хранения трех каналов.

2. Дискретное косинусное преобразование

Изображение разбивается на блоки 8x8 пикселей. Двумерное ДКП преобразует значения пикселей из пространственной области в частотную. Прямое преобразование вычисляется с помощью матричного умножения:

$$S = C^T * s * C$$

Элементы матрицы C определяются как:

$$C[i][j] = c[j] * \cos((2 * i + 1) * j * \pi / 16), \text{ где } i \in [0, 7], j \in [0, 7]$$

Коэффициент $c[j]$ равен:

$$c[j] = 1 / (2 * \sqrt{2}) \text{ при } j = 0$$

$$c[j] = 1 / 2 \text{ при } j \in [1, 7]$$

Обратное ДКП восстанавливает блок: $s = C * S * C^T$.

Временная сложность для блока 8x8 составляет $O(8^3) = O(512)$ операций при матричном умножении. Для всего изображения - $O(W * H * 8)$.

Пространственная сложность - $O(1)$ на блок, так как обработка выполняется поблочно.

3. Квантование

Процесс уменьшения точности коэффициентов ДКП для достижения сжатия. Каждый коэффициент $c[y][x]$ делится на соответствующий элемент матрицы квантования $q[y][x]$ и округляется:

$$c'[y][x] = \text{round}(c[y][x] / q[y][x])$$

Обратный процесс нормализации:

$$c[y][x] = c'[y][x] * q[y][x]$$

Временная сложность - $O(1)$ на коэффициент, $O(64)$ на блок.

Пространственная сложность - $O(1)$. Управление степенью сжатия осуществляется масштабированием таблицы квантования через параметр Quality.

4. Зигзаг-обход и энтропийное кодирование

После квантования в матрице 8×8 в левом верхнем углу концентрируется энергия, а ближе к правому нижнему - нули. Зигзаг-обход преобразует матрицу в одномерный вектор, группируя нули. Временная сложность - $O(64)$ на блок. Пространственная сложность - $O(64)$ для хранения вектора.

DC коэффициент кодируется разностно относительно DC коэффициента предыдущего блока. Временная сложность - $O(1)$ на блок.

AC коэффициенты кодируются с помощью RLE. Коды представляют собой пару: количество нулей и следующий элемент. Если далее только нули, используется код (0,0). Временная сложность RLE - $O(63)$ на блок.

Пространственная сложность - $O(63)$ в худшем случае.

5. Кодирование Хаффмана

Размеры значений DC разности и AC амплитуд кодируются кодами Хаффмана по таблицам стандарта ITU-T81. Временная сложность кодирования - $O(1)$ на символ при использовании предварительно построенных таблиц. Пространственная сложность - $O(1)$ для хранения таблиц фиксированного размера.

6. Downsampling и изменение размера

Применяется децимация каналов цветности Сб и Сг с коэффициентом 2. Это уменьшает объем данных в четыре раза по каждому каналу. Временная сложность - $O(W * H)$. Для масштабирования изображений используется билинейная интерполяция по четырем точкам. Линейная интерполяция для отрезка $[x_1, x_2]$ вычисляется по формуле:

$$y = y_1 + ((y_2 - y_1) / (x_2 - x_1)) * (x - x_1)$$

Временная сложность билинейной интерполяции - $O(W' * H')$, где W' и H' - новые размеры. Пространственная сложность - $O(W' * H')$ для хранения результата.

Общая временная сложность алгоритма сжатия составляет $O(W * H)$, так как каждый этап обрабатывает данные линейно или с постоянным множителем. Пространственная сложность определяется размером изображения и составляет $O(W * H)$.

Практическая часть

Входные данные.

В качестве тестовых данных используются следующие изображения: Lena.png, цветное изображение, изображение в оттенках серого, черно-белое изображение без дизеринга, черно-белое изображение с дизерингом.

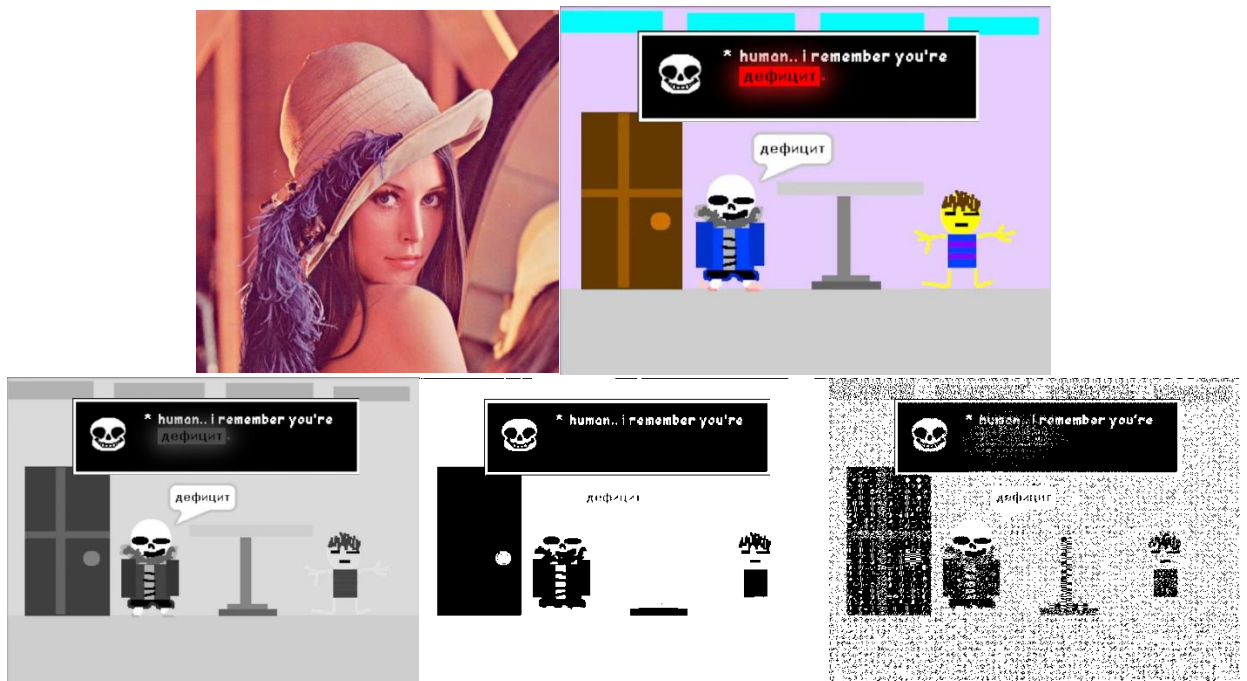


Рисунок 1 - Тестовые изображения.

Задание 1. Raw формат и оценка сжатия

Изображения переведены в собственный raw формат с метаданными о типе и размере. Выполнено сравнение размера raw файла с исходными форматами jpg и png, вычислен приблизительный коэффициент сжатия исходных форматов.

Image	Original (B)	RAW (B)	Ratio
Color	233,737	1,715,445	7.34
Grayscale	51,504	571,815	11.10
BW (no dither)	3,141	571,815	182.05
BW (dither)	50,323	571,815	11.36

Рисунок 2 - Таблица сравнения размеров файлов и коэффициентов сжатия для разных форматов.

Задание 2. Цветовые пространства

Реализованы функции перехода RGB в YCbCr и обратно. Визуальная проверка показала отсутствие видимых артефактов при прямом и обратном преобразовании. Метаданные raw формата дополнены информацией о цветовом пространстве.



Рисунок 3 - RGB -> YCbCr -> RGB.

Задание 3. Downsampling / Upsampling / Resizing

Реализована функция децимации. При апсемплинге наблюдается появление артефактов пикселизации при увеличении коэффициента децимации. Реализована функция линейной интерполяции, на ее основе - функция линейного сплайна. Реализована функция билинейной интерполяции для четырех точек.



Рисунок 4 – Downsampling x2 -> Upsampling x2.



Рисунок 5 - Downsampling x4 -> Upsampling x4.



Рисунок 6 - Resize.

Задание 4. Дискретное косинусное преобразование

Реализовано разбиение изображения на блоки 8x8. Для неполных краевых блоков реализовано дополнение усредненным значением пикселей блока. Реализовано прямое и обратное ДКП наивным методом и методом

матричного умножения. Проверка показала, что последовательное применение прямого и обратного ДКП возвращает исходное изображение с учетом погрешности вычислений. Реализованы функции квантования и нормализации.



Рисунок 7 - Исходное изображение и результат после прямого и обратного ДКП без квантования.

Задание 5. Зигзаг-обход и кодирование

Реализована функция зигзаг-обхода для квадратных матриц $N \times N$ и адаптированная версия для прямоугольных матриц $N \times M$. Реализовано разностное кодирование DC коэффициентов и RLE для AC коэффициентов с маркером конца блока (0,0) согласно стандарту ITU-T81.

Задание 6. Кодирование Хаффмана и параметр Quality

Реализовано кодирование переменной длины для DC разностей и AC амплитуд с использованием стандартных таблиц Хаффмана ITU-T81. Реализована адаптация таблицы квантования под параметр качества Quality по формулам:

$$S = 5000 / \text{Quality}, \text{ при } \text{Quality} \in [1, 50)$$

$$S = 200 - 2 * \text{Quality}, \text{ при } \text{Quality} \in [50, 100)$$

$$q'[y][x] = \text{ceil}((q[y][x] * S) / 100)$$

Задание 7. Запись и чтение файла

Реализованы функции сохранения сжатого потока данных в файл с учетом всех необходимых метаданных: цветовое пространство, размеры

изображения, таблицы квантования, таблицы Хаффмана. Реализованы функции чтения для последующей декомпрессии. Декомпрессия проходит успешно для всех тестовых файлов.

Задание 8. Зависимость размера от качества

Для каждого тестового изображения произведено сжатие с параметрами Quality от 10 до 100 с шагом не более 10. Построены графики зависимости размера сжатого файла от Quality.

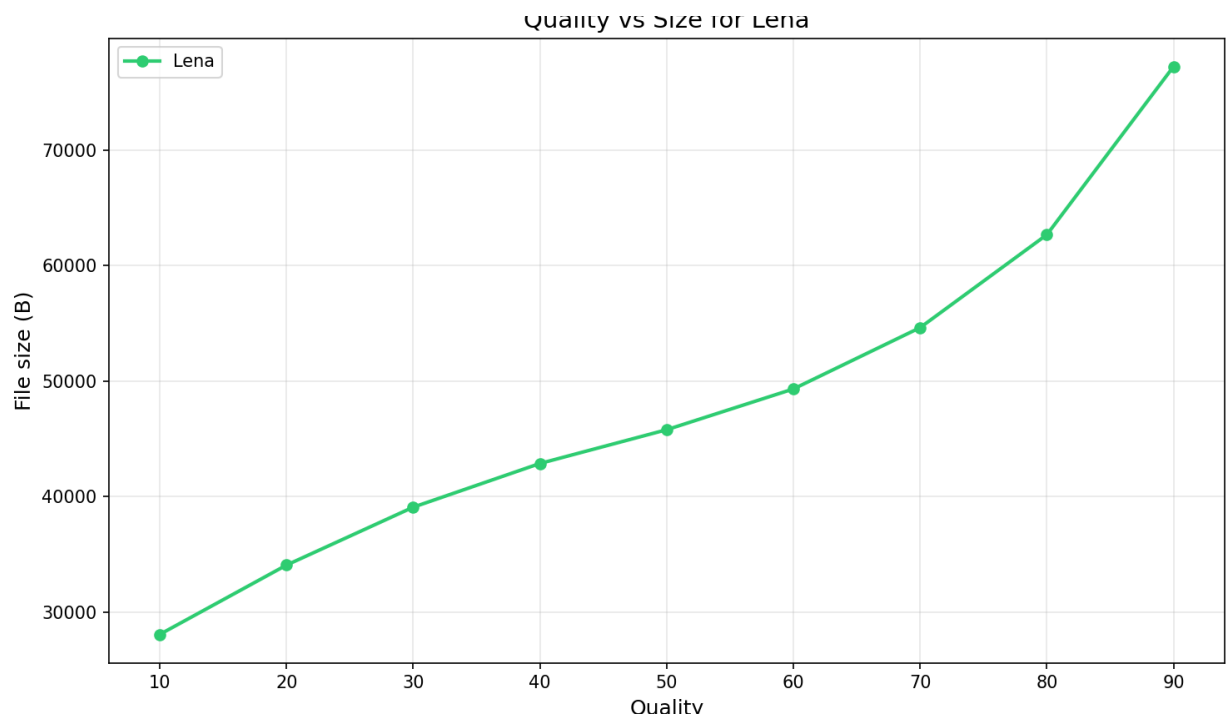


Рисунок 8 - График зависимости размера сжатого файла от Quality для Lena.png.

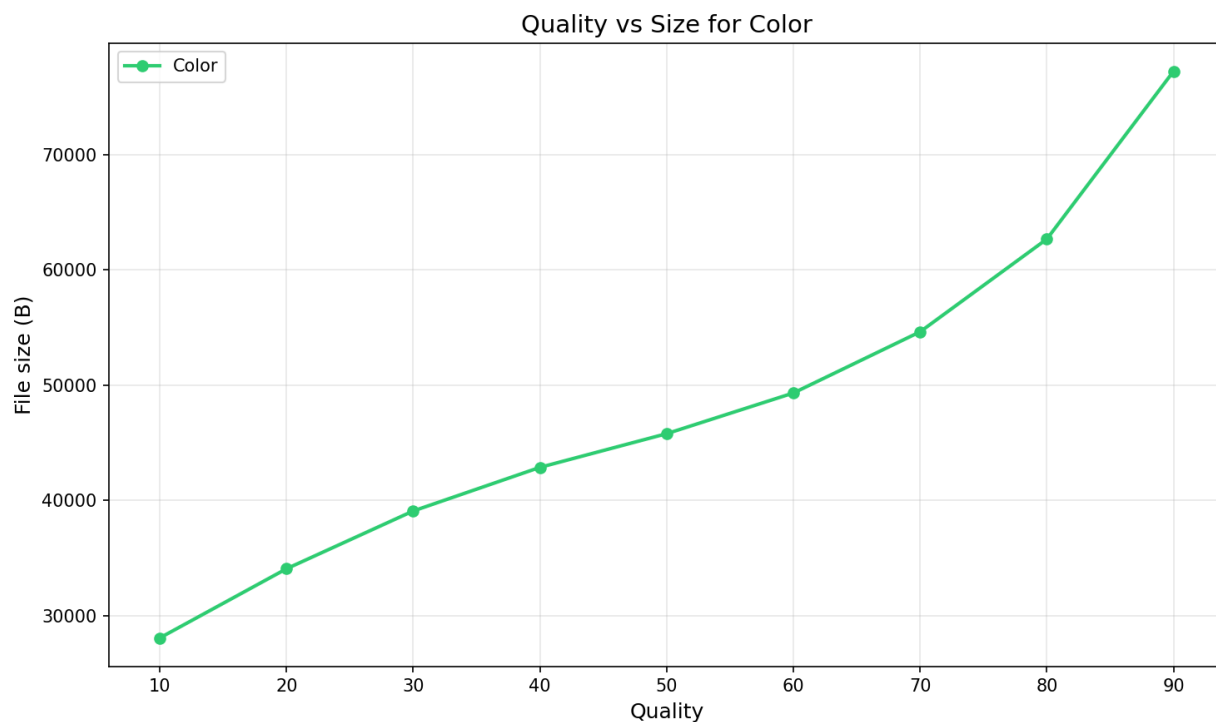


Рисунок 9 - График зависимости размера сжатого файла от Quality для цветного изображения.

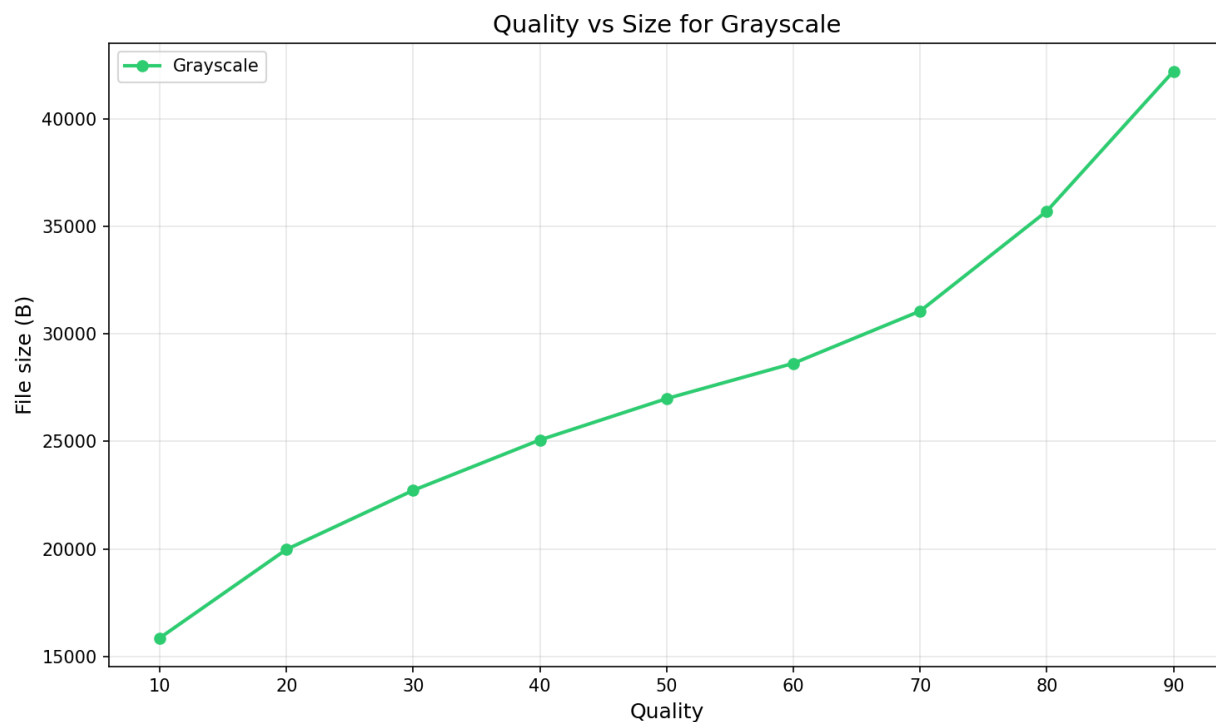


Рисунок 10 - График зависимости размера сжатого файла от Quality для изображения в оттенках серого.

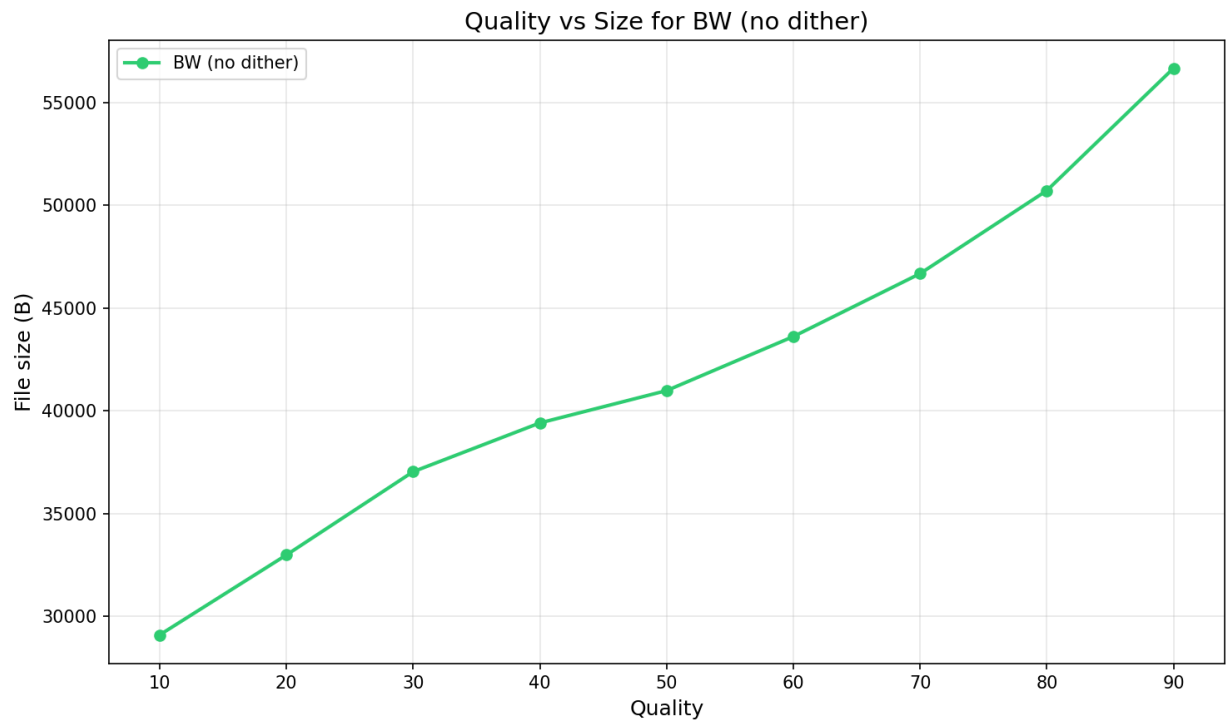


Рисунок 11 - График зависимости размера сжатого файла от Quality для черно-белого изображения без дизеринга.

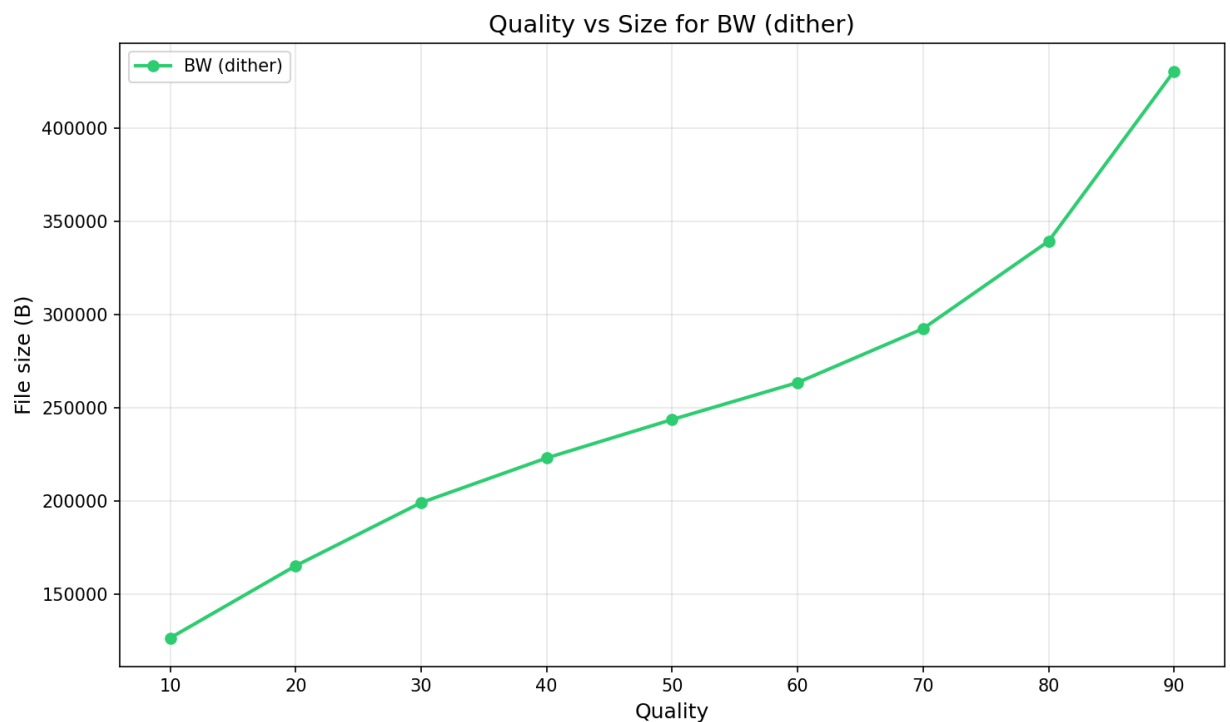


Рисунок 12 - График зависимости размера сжатого файла от Quality для черно-белого изображения с дизерингом.

Анализ графиков показывает, что с ростом параметра Quality размер выходного файла увеличивается, так как сохраняется больше

высокочастотных коэффициентов. Наибольший рост наблюдается на цветных и полутоновых изображениях.

Задание 9. Декомпрессия

Произведена декомпрессия сжатых изображений для значений Quality = 10, 50 и 90. Визуально подтверждена корректность работы алгоритма и проявление артефактов блочности на низких значениях Quality.



Рисунок 13 - Оригинал и декодированное изображение для Quality = 10.



Рисунок 14 - Оригинал и декодированное изображение для Quality = 50.



Рисунок 15 - Оригинал и декодированное изображение для Quality = 90.

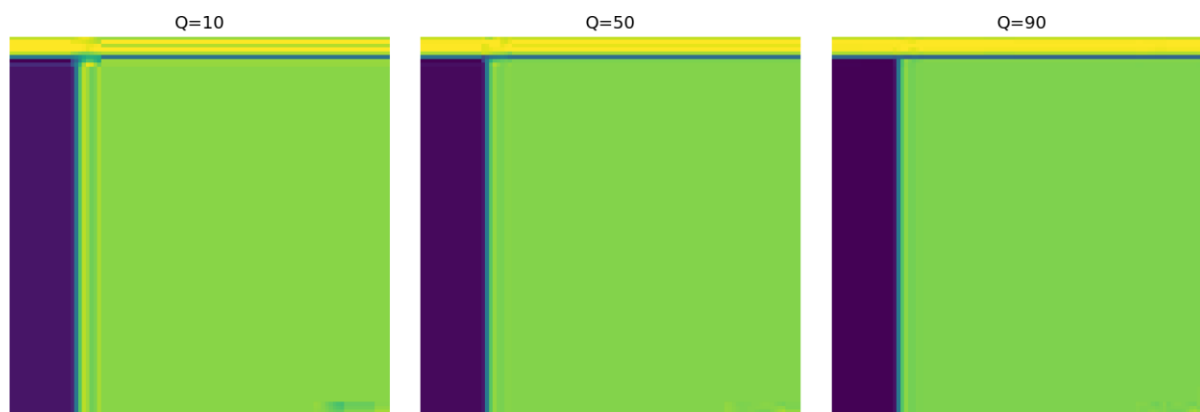


Рисунок 16 - Сравнение фрагментов изображений при разных Quality.

Вывод

В ходе работы реализован JPEG-подобный алгоритм сжатия изображений, включающий преобразование цветовых пространств RGB и YCbCr, дискретное косинусное преобразование блоков 8x8, квантование, зигзаг-обход, разностное кодирование DC коэффициентов, RLE для AC коэффициентов и кодирование Хаффмана согласно стандарту ITU-T81. Исследована зависимость размера сжатого файла от параметра Quality. Установлено, что при уменьшении Quality размер файла снижается, но появляются артефакты блочности. При высоких значениях Quality изображение визуально близко к оригиналу, однако степень сжатия падает. Реализованные функции записи и чтения метаданных обеспечивают корректное сохранение и восстановление сжатых изображений. Все поставленные задачи выполнены.

Ссылка на репозиторий Github

https://github.com/tethranialn/AISD_2.2_n2

Код программы

```
from PIL import Image
import os
import math
import json
import struct

def store_picture_as_bytes(picture, byte_file, color_mode=None):
    if picture.mode not in ['RGB', 'L', '1']:
        picture = picture.convert('RGB')

    if picture.mode == '1':
        pic_kind = 'bw'
        bytes_for_pixel = 1
        if color_mode is None:
            color_mode = 'BW'
    elif picture.mode == 'L':
        pic_kind = 'grayscale'
        bytes_for_pixel = 1
        if color_mode is None:
            color_mode = 'GRAY'
    else:
        pic_kind = 'color'
        bytes_for_pixel = 3
        if color_mode is None:
            color_mode = 'RGB'

    info_file = byte_file + '.info'
    with open(info_file, 'w', encoding='utf-8') as f:
        f.write(f'kind={pic_kind}\n')
        f.write(f'color_mode={color_mode}\n')
        f.write(f'w={picture.width}\n')
        f.write(f'h={picture.height}\n')
        f.write(f'bpp={bytes_for_pixel}\n')

    with open(byte_file, 'wb') as f:
        if picture.mode == 'RGB':
            for px in picture.get_flattened_data():
                f.write(bytes(px))
        else:
            for px in picture.get_flattened_data():
                if picture.mode == '1':
                    if px != 0:
                        px = 255
                f.write(bytes([px]))

    return {
        'kind': pic_kind,
        'w': picture.width,
        'h': picture.height,
        'bpp': bytes_for_pixel,
        'color_mode': color_mode
    }
```

```

def make_raw_copy(src_path, dst_path):
    pic = Image.open(src_path)
    src_bytes = os.path.getsize(src_path)
    info = store_picture_as_bytes(pic, dst_path)
    return {
        'src_bytes': src_bytes,
        'kind': info['kind'],
        'w': info['w'],
        'h': info['h'],
        'bpp': info['bpp']
    }

def show_compression_factor(data):
    raw_bytes = data['w'] * data['h'] * data['bpp']
    src_bytes = data['src_bytes']

    if src_bytes > 0:
        ratio = raw_bytes / src_bytes
        print(f'Source: {src_bytes} B')
        print(f'RAW: {raw_bytes} B')
        print(f'Ratio: {ratio:.2f}')
    else:
        print('Cannot compute ratio')

def change_colorspace_to_ycbcr(pic):
    pic = pic.convert('RGB')
    px_list = list(pic.get_flattened_data())
    new_px = []

    for (r_val, g_val, b_val) in px_list:
        y_val = 0.299 * r_val + 0.587 * g_val + 0.114 * b_val
        cb_val = -0.1687 * r_val - 0.3313 * g_val + 0.5 * b_val + 128
        cr_val = 0.5 * r_val - 0.4187 * g_val - 0.0813 * b_val + 128

        y_val = int(min(max(round(y_val), 0), 255))
        cb_val = int(min(max(round(cb_val), 0), 255))
        cr_val = int(min(max(round(cr_val), 0), 255))

        new_px.append((y_val, cb_val, cr_val))

    result = Image.new('RGB', pic.size)
    result.putdata(new_px)
    return result

def change_colorspace_to_rgb(pic):
    px_list = list(pic.get_flattened_data())
    new_px = []

    for (y_val, cb_val, cr_val) in px_list:
        r_val = y_val + 1.402 * (cr_val - 128)
        g_val = y_val - 0.344136 * (cb_val - 128) - 0.714136 * (cr_val - 128)
        b_val = y_val + 1.772 * (cb_val - 128)

        r_val = int(min(max(round(r_val), 0), 255))
        g_val = int(min(max(round(g_val), 0), 255))
        b_val = int(min(max(round(b_val), 0), 255))

        new_px.append((r_val, g_val, b_val))

```

```

result = Image.new('RGB', pic.size)
result.putdata(new_px)
return result

def shrink_picture(pic, step):
    pic = pic.convert('RGB')
    w, h = pic.size
    new_w = w // step
    new_h = h // step
    cut_w = new_w * step
    cut_h = new_h * step
    cropped = pic.crop((0, 0, cut_w, cut_h))
    small = Image.new('RGB', (new_w, new_h))
    for y in range(new_h):
        for x in range(new_w):
            small.putpixel((x, y), cropped.getpixel((x * step, y * step)))
    return small, (cut_w, cut_h)

def enlarge_picture(pic, step, target_size=None):
    pic = pic.convert('RGB')
    w, h = pic.size
    big_w = w * step
    big_h = h * step
    big = Image.new('RGB', (big_w, big_h))
    for y in range(h):
        for x in range(w):
            px = pic.getpixel((x, y))
            for dy in range(step):
                for dx in range(step):
                    big.putpixel((x * step + dx, y * step + dy), px)
    if target_size:
        big = big.crop((0, 0, target_size[0], target_size[1]))
    return big

def interpolate_linear(a1, a2, b1, b2, x):
    if a2 == a1:
        return b1
    return b1 + (b2 - b1) * (x - a1) / (a2 - a1)

def interpolate_spline(x_vals, y_vals, x):
    n = len(x_vals)
    if n != len(y_vals):
        raise ValueError('Arrays must match')
    if x <= x_vals[0]:
        return y_vals[0]
    if x >= x_vals[n - 1]:
        return y_vals[n - 1]
    for i in range(n - 1):
        if x_vals[i] <= x <= x_vals[i + 1]:
            return interpolate_linear(x_vals[i], x_vals[i + 1], y_vals[i], y_vals[i + 1], x)
    return None

def interpolate_bilinear(x1, x2, y1, y2, v11, v12, v21, v22, x, y):
    t1 = interpolate_linear(x1, x2, v11, v21, x)
    t2 = interpolate_linear(x1, x2, v12, v22, x)
    return interpolate_linear(y1, y2, t1, t2, y)

```

```

def find_interp_nodes(old_sz, idx, new_sz):
    if new_sz <= 0 or old_sz <= 0:
        raise ValueError('Sizes must be positive')
    if new_sz == 1:
        coord = 0.0
    else:
        coord = idx * (old_sz - 1) / (new_sz - 1)
    left = int(math.floor(coord))
    right = min(left + 1, old_sz - 1)
    return coord, left, right

def change_picture_size(pic, target_w, target_h):
    pic = pic.convert('RGB')
    w, h = pic.size
    if target_w <= 0 or target_h <= 0:
        raise ValueError('Target dimensions must be positive')
    result = Image.new('RGB', (target_w, target_h))
    for ny in range(target_h):
        y_coord, y1, y2 = find_interp_nodes(h, ny, target_h)
        for nx in range(target_w):
            x_coord, x1, x2 = find_interp_nodes(w, nx, target_w)
            c11 = pic.getpixel((x1, y1))
            c12 = pic.getpixel((x1, y2))
            c21 = pic.getpixel((x2, y1))
            c22 = pic.getpixel((x2, y2))
            r_val = interpolate_bilinear(x1, x2, y1, y2, c11[0], c12[0], c21[0],
c22[0], x_coord, y_coord)
            g_val = interpolate_bilinear(x1, x2, y1, y2, c11[1], c12[1], c21[1],
c22[1], x_coord, y_coord)
            b_val = interpolate_bilinear(x1, x2, y1, y2, c11[2], c12[2], c21[2],
c22[2], x_coord, y_coord)
            result.putpixel((nx, ny), (int(round(r_val)), int(round(g_val)),
int(round(b_val))))
    return result

def compute_alpha(idx, sz):
    return math.sqrt(1 / sz) if idx == 0 else math.sqrt(2 / sz)

def picture_to_2d_list(pic):
    pic = pic.convert('L')
    w, h = pic.size
    mat = []
    for y in range(h):
        row = []
        for x in range(w):
            row.append(pic.getpixel((x, y)))
        mat.append(row)
    return mat

def list2d_to_picture(mat):
    h = len(mat)
    w = len(mat[0])
    pic = Image.new('L', (w, h))
    for y in range(h):
        for x in range(w):
            val = int(round(mat[y][x]))

```



```

        val = min(max(val, 0), 255)
        pic.putpixel((x, y), val)
    return pic

def subtract_128(block):
    return [[float(v) - 128.0 for v in row] for row in block]

def add_128(block):
    return [[float(v) + 128.0 for v in row] for row in block]

def slice_into_pieces(mat, piece_h=8, piece_w=8, do_pad=True):
    h = len(mat)
    w = len(mat[0])
    pad_h = h
    pad_w = w

    if do_pad:
        if h % piece_h != 0:
            pad_h = h + (piece_h - h % piece_h)
        if w % piece_w != 0:
            pad_w = w + (piece_w - w % piece_w)

    padded = [[0.0 for _ in range(pad_w)] for _ in range(pad_h)]
    for y in range(h):
        for x in range(w):
            padded[y][x] = float(mat[y][x])

    pieces = []
    for by in range(0, pad_h, piece_h):
        row_pieces = []
        for bx in range(0, pad_w, piece_w):
            piece = []
            for y in range(piece_h):
                r = []
                for x in range(piece_w):
                    r.append(padded[by + y][bx + x])
                piece.append(r)
            row_pieces.append(piece)
        pieces.append(row_pieces)

    return pieces, h, w, pad_h, pad_w

def glue_pieces(pieces, orig_h, orig_w, piece_h=8, piece_w=8):
    pad_h = len(pieces) * piece_h
    pad_w = len(pieces[0]) * piece_w
    mat = [[0.0 for _ in range(pad_w)] for _ in range(pad_h)]

    for pi, prow in enumerate(pieces):
        for pj, piece in enumerate(prow):
            base_y = pi * piece_h
            base_x = pj * piece_w
            for y in range(piece_h):
                for x in range(piece_w):
                    mat[base_y + y][base_x + x] = piece[y][x]

    result = []
    for y in range(orig_h):
        result.append(mat[y][:orig_w])

```

```

    return result

def compute_dct_simple(block):
    n = len(block)
    m = len(block[0])
    coeffs = [[0.0 for _ in range(m)] for _ in range(n)]

    for u in range(n):
        for v in range(m):
            total = 0.0
            for x in range(n):
                for y in range(m):
                    total += (block[x][y] * math.cos((2 * x + 1) * u * math.pi / (2 *
n)) *
                                math.cos((2 * y + 1) * v * math.pi / (2 * m)))
            coeffs[u][v] = compute_alpha(u, n) * compute_alpha(v, m) * total
    return coeffs

def compute_idct_simple(coeffs):
    n = len(coeffs)
    m = len(coeffs[0])
    block = [[0.0 for _ in range(m)] for _ in range(n)]

    for x in range(n):
        for y in range(m):
            total = 0.0
            for u in range(n):
                for v in range(m):
                    total += (compute_alpha(u, n) * compute_alpha(v, m) *
coeffs[u][v] *
                                math.cos((2 * x + 1) * u * math.pi / (2 * n)) *
                                math.cos((2 * y + 1) * v * math.pi / (2 * m)))
            block[x][y] = total
    return block

def build_dct_matrix(sz):
    mat = [[0.0 for _ in range(sz)] for _ in range(sz)]
    for u in range(sz):
        for x in range(sz):
            mat[u][x] = compute_alpha(u, sz) * math.cos((2 * x + 1) * u * math.pi /
(2 * sz))
    return mat

def transpose_mat(mat):
    r = len(mat)
    c = len(mat[0])
    return [[mat[i][j] for i in range(r)] for j in range(c)]

def multiply_mats(a, b):
    ra = len(a)
    ca = len(a[0])
    rb = len(b)
    cb = len(b[0])
    if ca != rb:
        raise ValueError('Size mismatch')
    res = [[0.0 for _ in range(cb)] for _ in range(ra)]
    for i in range(ra):

```

```

        for j in range(cb):
            s = 0.0
            for k in range(ca):
                s += a[i][k] * b[k][j]
            res[i][j] = s
    return res

def compute_dct_matrix(block):
    n = len(block)
    m = len(block[0])
    if n != m:
        raise ValueError('Square blocks only')
    d = build_dct_matrix(n)
    dt = transpose_mat(d)
    return multiply_mats(multiply_mats(d, block), dt)

def compute_idct_matrix(coeffs):
    n = len(coeffs)
    m = len(coeffs[0])
    if n != m:
        raise ValueError('Square blocks only')
    d = build_dct_matrix(n)
    dt = transpose_mat(d)
    return multiply_mats(multiply_mats(dt, coeffs), d)

def apply_quantization(coeffs, q_mat):
    h = len(coeffs)
    w = len(coeffs[0])
    res = [[0 for _ in range(w)] for _ in range(h)]
    for y in range(h):
        for x in range(w):
            if q_mat[y][x] == 0:
                raise ValueError('Quantization matrix has zero')
            res[y][x] = round(coeffs[y][x] / q_mat[y][x])
    return res

def undo_quantization(q_coefs, q_mat):
    h = len(q_coefs)
    w = len(q_coefs[0])
    res = [[0.0 for _ in range(w)] for _ in range(h)]
    for y in range(h):
        for x in range(w):
            res[y][x] = q_coefs[y][x] * q_mat[y][x]
    return res

def run_dct_on_picture(mat, piece_h=8, piece_w=8, q_mat=None, use_matrix=False):
    pieces, orig_h, orig_w, _, _ = slice_into_pieces(mat, piece_h, piece_w,
do_pad=True)
    restored_pieces = []

    for prow in pieces:
        rrow = []
        for piece in prow:
            shifted = subtract_128(piece)
            if use_matrix:
                coeffs = compute_dct_matrix(shifted)
            else:

```

```

        coeffs = compute_dct_simple(shifted)

        if q_mat is not None:
            q_coeffs = apply_quantization(coeffs, q_mat)
            coeffs_to_use = undo_quantization(q_coeffs, q_mat)
        else:
            coeffs_to_use = coeffs

        if use_matrix:
            restored = compute_idct_matrix(coeffs_to_use)
        else:
            restored = compute_idct_simple(coeffs_to_use)

        rrow.append(add_128(restored))
        restored_pieces.append(rrow)

    return glue_pieces(restored_pieces, orig_h, orig_w, piece_h, piece_w)

def get_quant_table_luma():
    return [
        [16, 11, 10, 16, 24, 40, 51, 61],
        [12, 12, 14, 19, 26, 58, 60, 55],
        [14, 13, 16, 24, 40, 57, 69, 56],
        [14, 17, 22, 29, 51, 87, 80, 62],
        [18, 22, 37, 56, 68, 109, 103, 77],
        [24, 35, 55, 64, 81, 104, 113, 92],
        [49, 64, 78, 87, 103, 121, 120, 101],
        [72, 92, 95, 98, 112, 100, 103, 99],
    ]

def get_quant_table_chroma():
    return [
        [17, 18, 24, 47, 99, 99, 99, 99],
        [18, 21, 26, 66, 99, 99, 99, 99],
        [24, 26, 56, 99, 99, 99, 99, 99],
        [47, 66, 99, 99, 99, 99, 99, 99],
        [99, 99, 99, 99, 99, 99, 99, 99],
        [99, 99, 99, 99, 99, 99, 99, 99],
        [99, 99, 99, 99, 99, 99, 99, 99],
        [99, 99, 99, 99, 99, 99, 99, 99],
    ]

def zigzag_path(r, c):
    idx = []
    for s in range(r + c - 1):
        diag = []
        rs = max(0, s - (c - 1))
        re = min(r - 1, s)
        for rr in range(rs, re + 1):
            cc = s - rr
            diag.append((rr, cc))
        if s % 2 == 0:
            diag.reverse()
        idx.extend(diag)
    return idx

def scan_zigzag(mat):
    r = len(mat)

```

```

c = len(mat[0])
return [mat[rr][cc] for rr, cc in zigzag_path(r, c)]

def unscan_zigzag(vals, r, c):
    if len(vals) != r * c:
        raise ValueError('Wrong length')
    mat = [[0 for _ in range(c)] for _ in range(r)]
    for v, (rr, cc) in zip(vals, zigzag_path(r, c)):
        mat[rr][cc] = v
    return mat

def get_category(val):
    v = int(val)
    if v == 0:
        return 0
    return int(math.floor(math.log2(abs(v)))) + 1

def val_to_bitstring(v):
    v = int(v)
    sz = get_category(v)
    if sz == 0:
        return ''
    if v > 0:
        return format(v, f'0{sz}b')
    encoded = ((1 << sz) - 1) + v
    return format(encoded, f'0{sz}b')

def bitstring_to_val(bits, sz):
    if sz == 0:
        return 0
    if len(bits) != sz:
        raise ValueError('Wrong bit length')
    num = int(bits, 2)
    if bits[0] == '1':
        return num
    return num - ((1 << sz) - 1)

def diff_encode_dc(dc_list):
    if not dc_list:
        return []
    res = [int(dc_list[0])]
    for i in range(1, len(dc_list)):
        res.append(int(dc_list[i]) - int(dc_list[i - 1]))
    return res

def diff_decode_dc(diff_list):
    if not diff_list:
        return []
    res = [int(diff_list[0])]
    for i in range(1, len(diff_list)):
        res.append(res[-1] + int(diff_list[i]))
    return res

def rle_pack_ac(ac_vals):
    if len(ac_vals) != 63:

```

```

        raise ValueError('Need 63 AC coefficients')
    last = -1
    for i in range(62, -1, -1):
        if int(ac_vals[i]) != 0:
            last = i
            break
    if last == -1:
        return [(0, 0)]
    res = []
    zeros = 0
    for i in range(last + 1):
        v = int(ac_vals[i])
        if v == 0:
            zeros += 1
            if zeros == 16:
                res.append((15, 0))
                zeros = 0
        else:
            res.append((zeros, v))
            zeros = 0
    if last < 62:
        res.append((0, 0))
    return res

def rle_unpack_ac(packed):
    res = []
    for run, val in packed:
        run = int(run)
        val = int(val)
        if run == 0 and val == 0:
            while len(res) < 63:
                res.append(0)
            break
        if run == 15 and val == 0:
            res.extend([0] * 16)
            continue
        res.extend([0] * run)
        res.append(val)
        if len(res) > 63:
            raise ValueError('RLE overflow')
    while len(res) < 63:
        res.append(0)
    return res[:63]

def adjust_quant_table(base_table, q):
    if not (1 <= q < 100):
        raise ValueError('Quality must be in [1, 100]')
    if q < 50:
        s = 5000 / q
    else:
        s = 200 - 2 * q
    res = []
    for row in base_table:
        new_row = []
        for v in row:
            scaled = math.ceil((v * s) / 100.0)
            scaled = min(max(int(scaled), 1), 255)
            new_row.append(scaled)
        res.append(new_row)
    return res

```

```

def make_comparison_chart(raw_data, out_path):
    import matplotlib.pyplot as plt
    fig, ax = plt.subplots(figsize=(10, 4))
    ax.axis('tight')
    ax.axis('off')
    headers = ['Image', 'Original (B)', 'RAW (B)', 'Ratio']
    rows = []
    for nm, dat in raw_data.items():
        raw_sz = dat['w'] * dat['h'] * dat['bpp']
        ratio = raw_sz / dat['src_bytes'] if dat['src_bytes'] > 0 else 0
        rows.append([nm, f"{dat['src_bytes']:,}", f"{raw_sz:,}", f"{ratio:.2f}"])
    tbl = ax.table(cellText=rows, colLabels=headers, loc='center', cellLoc='center')
    tbl.auto_set_font_size(False)
    tbl.set_fontsize(10)
    tbl.scale(1.2, 1.5)
    for (i, j), cell in tbl.get_celld().items():
        if i == 0:
            cell.set_facecolor('#40466e')
            cell.set_text_props(weight='bold', color='white')
        elif i % 2 == 0:
            cell.set_facecolor('#f0f0f0')
    plt.tight_layout()
    plt.savefig(out_path, dpi=150)
    plt.close()

```

```

def make_quality_plot(results, out_path, ttl):
    import matplotlib.pyplot as plt
    fig, ax = plt.subplots(figsize=(10, 6))
    colors = ['#2ecc71', '#3498db', '#9b59b6', '#e74c3c', '#f39c12']
    markers = ['o', 's', '^', 'D', 'v']
    for (nm, szs), clr, mrk in zip(results.items(), colors, markers):
        q_vals = list(range(10, 100, 10))
        ax.plot(q_vals, szs, marker=mrk, linewidth=2, markersize=6, label=nm,
color=clr)
    ax.set_xlabel('Quality', fontsize=12)
    ax.set_ylabel('File size (B)', fontsize=12)
    ax.set_title(ttl, fontsize=14)
    ax.grid(True, alpha=0.3)
    ax.legend(loc='best')
    plt.tight_layout()
    plt.savefig(out_path, dpi=150)
    plt.close()

```

```

def make_side_by_side(orig, q, folder):
    tmp = os.path.join(folder, f'__tmp_q{q}.myjpg')
    compress_image_custom(orig, tmp, quality=q)
    restored, _ = decompress_image_custom(tmp)
    w, h = orig.size
    comp = Image.new('RGB', (w * 2, h))
    comp.paste(orig.convert('RGB'), (0, 0))
    comp.paste(restored.convert('RGB'), (w, 0))
    out = os.path.join(folder, f'side_by_side_q{q}.png')
    comp.save(out)
    os.remove(tmp)
    return out, restored

```

```

def make_dct_side_by_side(orig, restored, out_path):

```

```

w, h = orig.size
comp = Image.new('RGB', (w * 2, h))
comp.paste(orig.convert('RGB'), (0, 0))
comp.paste(restored.convert('RGB'), (w, 0))
comp.save(out_path)

def make_fragment_plot(imgs, out_path, box=(200, 200, 300, 300)):
    import matplotlib.pyplot as plt
    fig, axes = plt.subplots(1, len(imgs), figsize=(12, 4))
    for ax, (nm, img) in zip(axes, imgs.items()):
        frag = img.crop(box)
        ax.imshow(frag)
        ax.set_title(nm, fontsize=12)
        ax.axis('off')
    plt.tight_layout()
    plt.savefig(out_path, dpi=150)
    plt.close()

STD_LUMA_DC_BITS = [0, 1, 5, 1, 1, 1, 1, 1, 1, 0, 0, 0, 0, 0, 0]
STD_LUMA_DC_VALS = [0, 1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11]
STD_CHROMA_DC_BITS = [0, 3, 1, 1, 1, 1, 1, 1, 1, 1, 1, 0, 0, 0, 0]
STD_CHROMA_DC_VALS = [0, 1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11]
STD_LUMA_AC_BITS = [0, 2, 1, 3, 3, 2, 4, 3, 5, 5, 4, 4, 0, 0, 1, 125]
STD_LUMA_AC_VALS = [
    0x01, 0x02, 0x03, 0x00, 0x04, 0x11, 0x05, 0x12, 0x21, 0x31, 0x41, 0x06, 0x13,
    0x51, 0x61,
    0x07, 0x22, 0x71, 0x14, 0x32, 0x81, 0x91, 0xA1, 0x08, 0x23, 0x42, 0xB1, 0xC1,
    0x15, 0x52,
    0xD1, 0xF0, 0x24, 0x33, 0x62, 0x72, 0x82, 0x09, 0x0A, 0x16, 0x17, 0x18, 0x19,
    0x1A, 0x25,
    0x26, 0x27, 0x28, 0x29, 0x2A, 0x34, 0x35, 0x36, 0x37, 0x38, 0x39, 0x3A, 0x43,
    0x44, 0x45,
    0x46, 0x47, 0x48, 0x49, 0x4A, 0x53, 0x54, 0x55, 0x56, 0x57, 0x58, 0x59, 0x5A,
    0x63, 0x64,
    0x65, 0x66, 0x67, 0x68, 0x69, 0x6A, 0x73, 0x74, 0x75, 0x76, 0x77, 0x78, 0x79,
    0x7A, 0x83,
    0x84, 0x85, 0x86, 0x87, 0x88, 0x89, 0x8A, 0x92, 0x93, 0x94, 0x95, 0x96, 0x97,
    0x98, 0x99,
    0x9A, 0xA2, 0xA3, 0xA4, 0xA5, 0xA6, 0xA7, 0xA8, 0xA9, 0xAA, 0xB2, 0xB3, 0xB4,
    0xB5, 0xB6,
    0xB7, 0xB8, 0xB9, 0xBA, 0xC2, 0xC3, 0xC4, 0xC5, 0xC6, 0xC7, 0xC8, 0xC9, 0xCA,
    0xD2, 0xD3,
    0xD4, 0xD5, 0xD6, 0xD7, 0xD8, 0xD9, 0xDA, 0xE1, 0xE2, 0xE3, 0xE4, 0xE5, 0xE6,
    0xE7, 0xE8,
    0xE9, 0xEA, 0xF1, 0xF2, 0xF3, 0xF4, 0xF5, 0xF6, 0xF7, 0xF8, 0xF9, 0xFA
]
STD_CHROMA_AC_BITS = [0, 2, 1, 2, 4, 4, 3, 4, 7, 5, 4, 4, 0, 1, 2, 119]
STD_CHROMA_AC_VALS = [
    0x00, 0x01, 0x02, 0x03, 0x11, 0x04, 0x05, 0x21, 0x31, 0x06, 0x12, 0x41, 0x51,
    0x07, 0x61,
    0x71, 0x13, 0x22, 0x32, 0x81, 0x08, 0x14, 0x42, 0x91, 0xA1, 0xB1, 0xC1, 0x09,
    0x23, 0x33,
    0x52, 0xF0, 0x15, 0x62, 0x72, 0xD1, 0x0A, 0x16, 0x24, 0x34, 0xE1, 0x25, 0xF1,
    0x17, 0x18,
    0x19, 0x1A, 0x26, 0x27, 0x28, 0x29, 0x2A, 0x35, 0x36, 0x37, 0x38, 0x39, 0x3A,
    0x43, 0x44,
    0x45, 0x46, 0x47, 0x48, 0x49, 0x4A, 0x53, 0x54, 0x55, 0x56, 0x57, 0x58, 0x59,
    0x5A, 0x63,
    0x64, 0x65, 0x66, 0x67, 0x68, 0x69, 0x6A, 0x73, 0x74, 0x75, 0x76, 0x77, 0x78,
    0x79, 0x7A,

```



```

    0x82, 0x83, 0x84, 0x85, 0x86, 0x87, 0x88, 0x89, 0x8A, 0x92, 0x93, 0x94, 0x95,
    0x96, 0x97,
    0x98, 0x99, 0x9A, 0xA2, 0xA3, 0xA4, 0xA5, 0xA6, 0xA7, 0xA8, 0xA9, 0xAA, 0xB2,
    0xB3, 0xB4,
    0xB5, 0xB6, 0xB7, 0xB8, 0xB9, 0xBA, 0xC2, 0xC3, 0xC4, 0xC5, 0xC6, 0xC7, 0xC8,
    0xC9, 0xCA,
    0xD2, 0xD3, 0xD4, 0xD5, 0xD6, 0xD7, 0xD8, 0xD9, 0xDA, 0xE2, 0xE3, 0xE4, 0xE5,
    0xE6, 0xE7,
    0xE8, 0xE9, 0xEA, 0xF2, 0xF3, 0xF4, 0xF5, 0xF6, 0xF7, 0xF8, 0xF9, 0xFA
]

```

```

def build_huffman_tables(bits, vals):
    code = 0
    k = 0
    enc = {}
    dec = {}
    for bl, cnt in enumerate(bits, start=1):
        for _ in range(cnt):
            sym = vals[k]
            bit_str = format(code, f'0{bl}b')
            enc[sym] = bit_str
            dec[bit_str] = sym
            code += 1
            k += 1
        code <= 1
    return enc, dec

```

```

LUMA_DC_ENC, LUMA_DC_DEC = build_huffman_tables(STD_LUMA_DC_BITS, STD_LUMA_DC_VALS)
CHROMA_DC_ENC, CHROMA_DC_DEC = build_huffman_tables(STD_CHROMA_DC_BITS,
STD_CHROMA_DC_VALS)
LUMA_AC_ENC, LUMA_AC_DEC = build_huffman_tables(STD_LUMA_AC_BITS, STD_LUMA_AC_VALS)
CHROMA_AC_ENC, CHROMA_AC_DEC = build_huffman_tables(STD_CHROMA_AC_BITS,
STD_CHROMA_AC_VALS)

```

```

def encode_dc_symbol(diff, dc_enc):
    sz = get_category(diff)
    return dc_enc[sz] + val_to_bitstring(diff)

```

```

def encode_ac_block(ac_vals, ac_enc):
    bits = ''
    packed = rle_pack_ac(ac_vals)
    for run, val in packed:
        if run == 0 and val == 0:
            bits += ac_enc[0x00]
            continue
        if run == 15 and val == 0:
            bits += ac_enc[0xF0]
            continue
        sz = get_category(val)
        amp_bits = val_to_bitstring(val)
        sym = (run << 4) | sz
        bits += ac_enc[sym] + amp_bits
    return bits, packed

```

```

class BitStreamWriter:
    def __init__(self):
        self._bits = ''

```

```

def put(self, s):
    self._bits += s

def to_bytes(self):
    padded = self._bits
    while len(padded) % 8 != 0:
        padded += '1'
    data = bytearray()
    for i in range(0, len(padded), 8):
        b = int(padded[i:i + 8], 2)
        data.append(b)
        if b == 0xFF:
            data.append(0x00)
    return bytes(data), len(self._bits)

class BitStreamReader:
    def __init__(self, data, valid_len):
        unstuffed = bytearray()
        i = 0
        while i < len(data):
            b = data[i]
            unstuffed.append(b)
            if b == 0xFF and i + 1 < len(data) and data[i + 1] == 0x00:
                i += 2
            else:
                i += 1
        self._bits = ''.join(format(b, '08b') for b in unstuffed)[:valid_len]
        self._pos = 0

    def take(self, cnt):
        if self._pos + cnt > len(self._bits):
            raise ValueError('Not enough bits')
        res = self._bits[self._pos:self._pos + cnt]
        self._pos += cnt
        return res

def decode_huff_sym(reader, dec_table):
    cur = ''
    while True:
        cur += reader.take(1)
        if cur in dec_table:
            return dec_table[cur]
        if len(cur) > 16:
            raise ValueError('Cannot decode')

def decode_dc_val(reader, dc_dec):
    sz = decode_huff_sym(reader, dc_dec)
    if sz == 0:
        return 0
    bits = reader.take(sz)
    return bitstring_to_val(bits, sz)

def decode_ac_block(reader, ac_dec):
    res = []
    while len(res) < 63:
        sym = decode_huff_sym(reader, ac_dec)
        if sym == 0x00:

```

```

        res.extend([0] * (63 - len(res)))
        break
    if sym == 0xF0:
        res.extend([0] * 16)
        continue
    run = (sym >> 4) & 0x0F
    sz = sym & 0x0F
    res.extend([0] * run)
    bits = reader.take(sz)
    res.append(bitstring_to_val(bits, sz))
    if len(res) > 63:
        raise ValueError('AC decode error')
return res[:63]

```

```

def pack_one_block(block, prev_dc, q_mat, dc_enc, ac_enc):
    shifted = subtract_128(block)
    coeffs = compute_dct_simple(shifted)
    q_coeffs = apply_quantization(coeffs, q_mat)
    zz = [int(x) for x in scan_zigzag(q_coeffs)]
    dc_val = zz[0]
    dc_diff = dc_val - prev_dc
    dc_bits = encode_dc_symbol(dc_diff, dc_enc)
    ac_bits, _ = encode_ac_block(zz[1:], ac_enc)
    return {'bits': dc_bits + ac_bits, 'dc': dc_val, 'zz': zz, 'q': q_coeffs}

```

```

def unpack_one_block(reader, prev_dc, q_mat, dc_dec, ac_dec):
    dc_diff = decode_dc_val(reader, dc_dec)
    dc_val = prev_dc + dc_diff
    ac_vals = decode_ac_block(reader, ac_dec)
    zz = [dc_val] + ac_vals
    q_coeffs = unscan_zigzag(zz, 8, 8)
    coeffs = undo_quantization(q_coeffs, q_mat)
    block = add_128(compute_idct_simple(coeffs))
    return block, dc_val

```

```

def pic_channel_to_mat(ch):
    w, h = ch.size
    mat = []
    for y in range(h):
        row = []
        for x in range(w):
            row.append(int(ch.getpixel((x, y))))
        mat.append(row)
    return mat

```

```

def mat_to_pic_channel(mat):
    h = len(mat)
    w = len(mat[0])
    ch = Image.new('L', (w, h))
    for y in range(h):
        for x in range(w):
            v = int(round(mat[y][x]))
            v = min(max(v, 0), 255)
            ch.putpixel((x, y), v)
    return ch

```

```

def split_pic_to_ycbcr(pic):

```

```

    if pic.mode == 'L':
        return {'Y': pic.convert('L')}, 'GRAY'
    ycbcr = pic.convert('YCbCr')
    y, cb, cr = ycbcr.split()
    return {'Y': y, 'Cb': cb, 'Cr': cr}, 'YCbCr'

def merge_ycbcr_to_pic(ch_dict, cs):
    if cs == 'GRAY':
        return ch_dict['Y'].convert('L')
    merged = Image.merge('YCbCr', (ch_dict['Y'], ch_dict['Cb'], ch_dict['Cr']))
    return merged.convert('RGB')

def compress_image_custom(src, dst, quality=50):
    if isinstance(src, str):
        pic = Image.open(src)
    else:
        pic = src
    if pic.mode not in ['RGB', 'L']:
        pic = pic.convert('RGB')
    ch_dict, cs = split_pic_to_ycbcr(pic)
    q_luma = adjust_quant_table(get_quant_table_luma(), quality)
    q_chroma = adjust_quant_table(get_quant_table_chroma(), quality)
    writer = BitStreamWriter()
    meta = {
        'magic': 'MYJPEG1', 'w': pic.width, 'h': pic.height,
        'q': quality, 'cs': cs,
        'q_tables': {'Y': q_luma, 'Cb': q_chroma, 'Cr': q_chroma},
        'comps': []
    }
    for nm, ch in ch_dict.items():
        mat = pic_channel_to_mat(ch)
        pieces, oh, ow, ph, pw = slice_into_pieces(mat, 8, 8, do_pad=True)
        comp = {'nm': nm, 'orig_sz': [ow, oh], 'pad_sz': [pw, ph],
                'blocks_w': len(pieces[0]), 'blocks_h': len(pieces)}
        if nm == 'Y':
            dc_enc, ac_enc = LUMA_DC_ENC, LUMA_AC_ENC
            q_mat = q_luma
        else:
            dc_enc, ac_enc = CHROMA_DC_ENC, CHROMA_AC_ENC
            q_mat = q_chroma
        prev = 0
        for prow in pieces:
            for piece in prow:
                enc = pack_one_block(piece, prev, q_mat, dc_enc, ac_enc)
                writer.put(enc['bits'])
                prev = enc['dc']
            meta['comps'].append(comp)
    payload, vlen = writer.to_bytes()
    meta['vlen'] = vlen
    meta_json = json.dumps(meta, ensure_ascii=False).encode('utf-8')
    with open(dst, 'wb') as f:
        f.write(b'MYJPEG1')
        f.write(struct.pack('>I', len(meta_json)))
        f.write(meta_json)
        f.write(payload)
    return {'meta': meta, 'file_sz': os.path.getsize(dst)}

def decompress_image_custom(src):
    with open(src, 'rb') as f:

```

```

        magic = f.read(7)
        if magic != b'MYJPEG1':
            raise ValueError('Bad format')
        meta_len = struct.unpack('>I', f.read(4))[0]
        meta = json.loads(f.read(meta_len).decode('utf-8'))
        payload = f.read()
    reader = BitStreamReader(payload, meta['vlen'])
    restored = {}
    for comp in meta['comps']:
        nm = comp['nm']
        ow, oh = comp['orig_sz']
        bw, bh = comp['blocks_w'], comp['blocks_h']
        if nm == 'Y':
            q_mat = meta['q_tables']['Y']
            dc_dec, ac_dec = LUMA_DC_DEC, LUMA_AC_DEC
        else:
            q_mat = meta['q_tables'][nm]
            dc_dec, ac_dec = CHROMA_DC_DEC, CHROMA_AC_DEC
        pieces = []
        prev = 0
        for _ in range(bh):
            row = []
            for _ in range(bw):
                block, prev = unpack_one_block(reader, prev, q_mat, dc_dec, ac_dec)
                row.append(block)
            pieces.append(row)
        mat = glue_pieces(pieces, oh, ow, 8, 8)
        restored[nm] = mat_to_pic_channel(mat)
    return merge_ycbcr_to_pic(restored, meta['cs']), meta

def main():
    folder = r"C:\.Study\AISD\AISD_2.2_n2"
    os.makedirs(folder, exist_ok=True)
    src_pic = os.path.join(folder, "image.png")
    color = Image.open(src_pic).convert('RGB')
    gray = color.convert('L')
    bw_simple = gray.convert('1', dither=Image.Dither.NONE)
    bw_fancy = color.convert('1')
    gray.save(os.path.join(folder, 'grayscale.png'))
    bw_simple.save(os.path.join(folder, 'bw_rounded.png'))
    bw_fancy.save(os.path.join(folder, 'bw_dither.png'))

    info_c = make_raw_copy(src_pic, os.path.join(folder, 'image.raw'))
    info_g = make_raw_copy(os.path.join(folder, 'grayscale.png'),
os.path.join(folder, 'grayscale.raw'))
    info_b1 = make_raw_copy(os.path.join(folder, 'bw_rounded.png'),
os.path.join(folder, 'bw_rounded.raw'))
    info_b2 = make_raw_copy(os.path.join(folder, 'bw_dither.png'),
os.path.join(folder, 'bw_dither.raw'))

    make_comparison_chart({
        'Color': info_c, 'Grayscale': info_g,
        'BW (no dither)': info_b1, 'BW (dither)': info_b2
    }, os.path.join(folder, 'raw_comparison.png'))

    ycbcr = change_colorspace_to_ycbcr(color)
    ycbcr.save(os.path.join(folder, 'ycbcr.png'))
    rgb_again = change_colorspace_to_rgb(ycbcr)
    rgb_again.save(os.path.join(folder, 'rgb_restored.png'))
    store_picture_as_bytes(ycbcr, os.path.join(folder, 'ycbcr.raw'), 'YCbCr')

```

```

small2, _ = shrink_picture(color, 2)
small2.save(os.path.join(folder, 'down_x2.png'))
big2 = enlarge_picture(small2, 2, color.size)
big2.save(os.path.join(folder, 'up_x2.png'))

small4, _ = shrink_picture(color, 4)
small4.save(os.path.join(folder, 'downsample_x4.png'))
big4 = enlarge_picture(small4, 4, color.size)
big4.save(os.path.join(folder, 'upsample_from_x4.png'))

half = change_picture_size(color, color.width // 2, color.height // 2)
half.save(os.path.join(folder, 'resize_half_bilinear.png'))
back = change_picture_size(half, color.width, color.height)
back.save(os.path.join(folder, 'resize_back_bilinear.png'))

gray_mat = picture_to_2d_list(gray)
restored_mat = run_dct_on_picture(gray_mat, 8, 8, q_mat=None, use_matrix=False)
restored_pic = list2d_to_picture(restored_mat)
restored_pic.save(os.path.join(folder, 'dct_restored.png'))
make_dct_side_by_side(gray, restored_pic, os.path.join(folder,
'dct_comparison.png'))

imgs_for_plot = {'Lena': color, 'Color': color, 'Grayscale': gray,
                  'BW_no_dither': bw_simple, 'BW_dither': bw_fancy}
all_sizes = {}
q_vals = list(range(10, 100, 10))
saved_q10 = saved_q50 = saved_q90 = None

for nm, img in imgs_for_plot.items():
    sz_list = []
    for q in q_vals:
        tmp = os.path.join(folder, f'__tmp_{nm}_q{q}.myjpg')
        info = compress_image_custom(img, tmp, quality=q)
        sz_list.append(info['file_sz'])
        if q in [10, 50, 90] and nm == 'Grayscale':
            restored_img, _ = decompress_image_custom(tmp)
            if q == 10:
                saved_q10 = restored_img
            elif q == 50:
                saved_q50 = restored_img
            elif q == 90:
                saved_q90 = restored_img
        os.remove(tmp)
    all_sizes[nm] = sz_list

    make_quality_plot({'Lena': all_sizes['Lena']}, os.path.join(folder,
'quality_lena.png'), 'Quality vs Size for Lena')
    make_quality_plot({'Color': all_sizes['Color']}, os.path.join(folder,
'quality_color.png'), 'Quality vs Size for Color')
    make_quality_plot({'Grayscale': all_sizes['Grayscale']}, os.path.join(folder,
'quality_gray.png'), 'Quality vs Size for Grayscale')
    make_quality_plot({'BW (no dither)': all_sizes['BW_no_dither']},
os.path.join(folder, 'quality_bw_no_dither.png'), 'Quality vs Size for BW (no
dither)')
    make_quality_plot({'BW (dither)': all_sizes['BW_dither']}, os.path.join(folder,
'quality_bw_dither.png'), 'Quality vs Size for BW (dither)')

make_side_by_side(gray, 10, folder)
make_side_by_side(gray, 50, folder)
make_side_by_side(gray, 90, folder)

if saved_q10 and saved_q50 and saved_q90:

```

```
make_fragment_plot({'Q=10': saved_q10, 'Q=50': saved_q50, 'Q=90': saved_q90},
                   os.path.join(folder, 'fragments_comparison.png'))

compress_image_custom(gray, os.path.join(folder, 'compressed.myjpg'), quality=50)
decompressed, _ = decompress_image_custom(os.path.join(folder,
'compressed.myjpg'))
decompressed.save(os.path.join(folder, 'decompressed.png'))

if __name__ == '__main__':
    main()
```